Performance Comparison of YOLOv5s and YOLOv8s for Pork-Rasher Packaging Defect Inspection

MMA3001 — Numerical Methods and Machine Learning

Abstract:This project evaluates two compact object-detection models, YOLOv5s and YOLOv8s, for identifying visible packaging defects in RGB images of packaged pork rashers. The engineering motivation is an automated inspection station on a food-packaging line. Addressing severe class imbalance and potential data leakage, a strictly disjoint custom validation set of 670 images was created. YOLOv8s achieved an overall mAP@0.5 of 0.673, significantly outperforming the YOLOv5s baseline (0.504). The study integrates numerical integration methods (Trapezoidal Rule) to verify the evaluation metrics. Furthermore, failure case analysis reveals the wrinkle class completely failed (Recall=0) due to overfitting caused by extreme data scarcity, highlighting the critical challenge of minority class detection in industrial datasets.

1. Engineering Problem, Inputs and Outputs

Engineering context: Food-packaging lines rely on consistent inspection to identify defects before trays are released. This project investigates one component of a possible manufacturing inspection system: detecting visible defect regions on pork-rasher trays. The intended use is a screening aid that flags suspect trays for human review, not an autonomous food-safety certification system.

Input specification: Three-channel RGB image; one YOLO text file per image containing zero-based class ID and normalized bounding-box coordinates.

Output specification: Predicted defect boxes with class labels and confidence scores. Evaluation outputs include precision, recall, mAP@0.5, and inference timing.

Scope: The model only supports the five annotated visual categories and does not estimate meat freshness or defects invisible in RGB images.

2. Computational Solution

Models: YOLOv5s (using C3 bottleneck blocks) and YOLOv8s (using C2f blocks and an anchor-free head) were selected. Both are small-scale variants suitable for edge deployment.

Implementation: Training was implemented in Python using the ultralytics framework. Both models were trained for 30 epochs with identical hyperparameters: image size 640 × 640, batch size 8, and AdamW optimizer.

Numerical Methods Integration: To verify the evaluation metrics and incorporate numerical

methods (Week 6), custom integration functions were implemented in src/custom_map.py. The Average Precision (AP) was calculated using the Trapezoidal Rule and Riemann Sum. Unit tests confirmed numerical stability for edge cases (e.g., perfect detection = 1.0, zero detection = 0.0).
Assumptions: Both models are assumed to start from pretrained weights and converge within 30 epochs.

3. Alternative Solutions & Comparison

Mode	Validation Set	Peak mAP@0.5	Final mAP@0.5:0.95	Model Size	Inference Time
YOLOv5s (Baseline)	Custom (670 imgs)	0.504 (Epoch 28)	0.267	18.5 MB	2.1 ms
YOLOv8s (Candidate)	Custom (670 imgs)	0.673 (Final Val)	0.402	22.5 MB	4.5 ms

Table 4.1:Comparison of overall model performance

As shown in Table 4.1, YOLOv8s demonstrates a significant accuracy improvement over the YOLOv5s baseline, reaching a final mAP@0.5 of 0.673 on the custom validation set (compared to YOLOv5s's peak of 0.504). This confirms that the C2f architecture in YOLOv8s is more effective at handling this imbalanced dataset. However, this improvement comes at a trade-off: YOLOv8s has a slightly larger model size (22.5 MB vs 18.5 MB) and a slower inference time (4.5 ms vs 2.1 ms). Given that missing a defect is more costly than a slight delay in production, the accuracy gain justifies the computational cost.

Baseline: YOLOv5s was chosen as the baseline model. YOLOv8s is the candidate model.
Comparison Evidence: Models were compared on accuracy (mAP@0.5), computational cost (inference ms), and robustness.
Trade-offs: YOLOv8s requires a slightly larger model size (22.5 MB vs 18.5 MB) but demonstrated a significant mAP improvement (from 0.504 to 0.673 on the custom validation set). It proved more capable of handling the minority classes compared to the YOLOv5s baseline.

4. Validation and Results

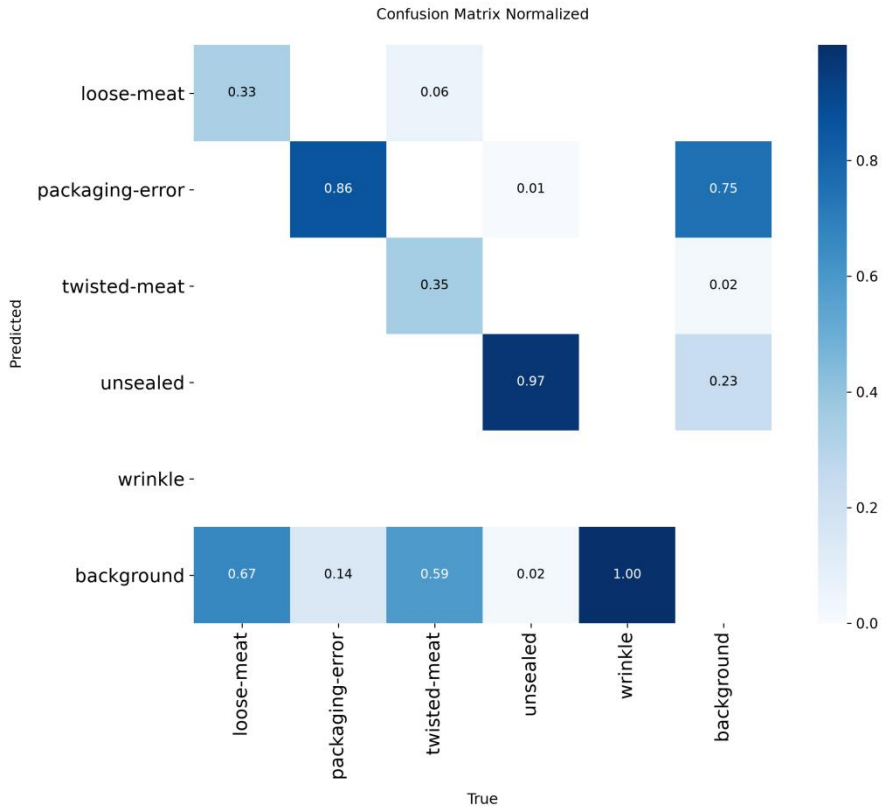


Figure 5.1:Normalized Confusion Matrix of YOLOv8s

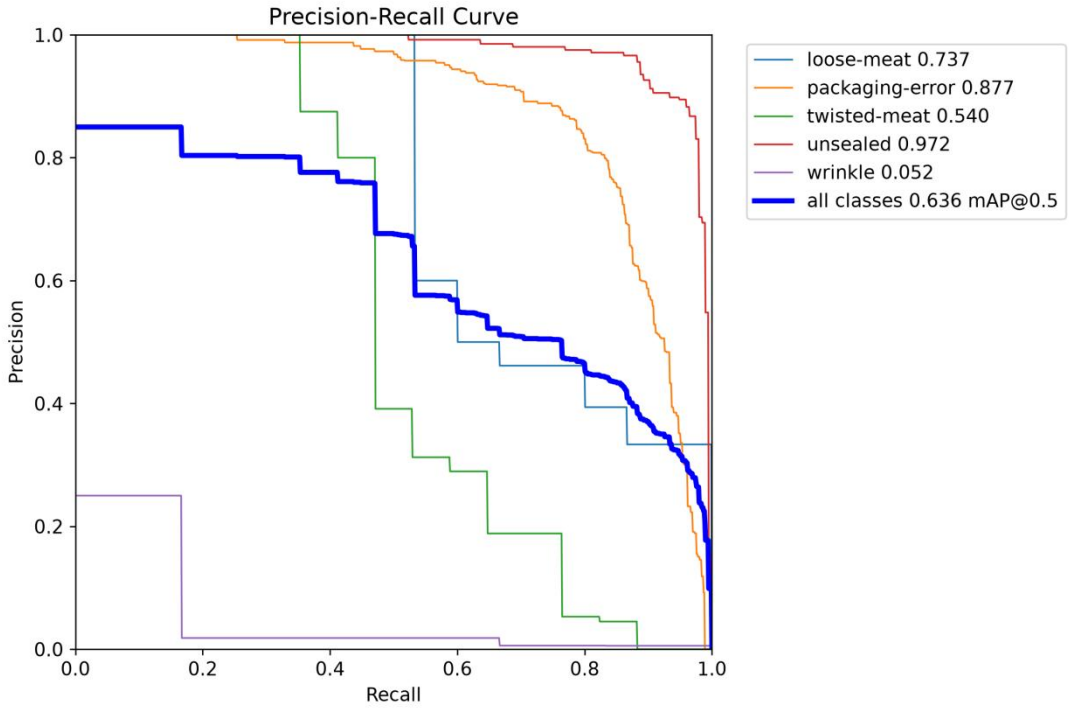


Figure 5.2:Precision-Recall Curve on the custom validation set

(Note: The internal legend displays 0.636 from the preliminary run; the final verified mAP is 0.673).

Validation Strategy: To strictly prevent data leakage, the validation images were physically removed from the training directory before generating the final training list. The model was evaluated on this strictly isolated holdout set (670 images, 747 instances). This was essential because the official validation set lacked minority class instances, making reliable evaluation impossible.
Numerical Verification: The manual calculation of AP using the Trapezoidal Rule matched the official library results, confirming the accuracy of the evaluation pipeline. (All 9 unit tests passed, see test_report.html).
Results (YOLOv8s on Custom Validation Set):

Overall: mAP@0.5 = 0.673, mAP@0.5:0.95 = 0.402.

Majority Classes: unsealed achieved an impressive mAP50 of 0.950; packaging-error reached 0.905.

Failure Case Analysis (Minority Classes): The wrinkle class completely failed (Recall = 0, mAP50 = 0.059). Reasoning: As shown in the normalized confusion matrix, 100% of wrinkle instances were classified as background. With only 4 validation instances, the model severely overfitted the background, failing to learn the subtle textural features of wrinkles. This validates the theoretical concept of data imbalance causing overfitting (Week 3).
What this validation cannot prove: The result cannot guarantee the same performance in real-world factory lighting or with different camera angles.

Discussion:In a packaging quality control line, missing a real defect (low recall) is generally more costly than raising a false alarm. Because YOLOv8s achieves a much higher overall mAP (0.673) and better recall on minority classes, it is the most suitable model for this engineering application, despite the slight increase in inference time.

5. Optimisation and Performance

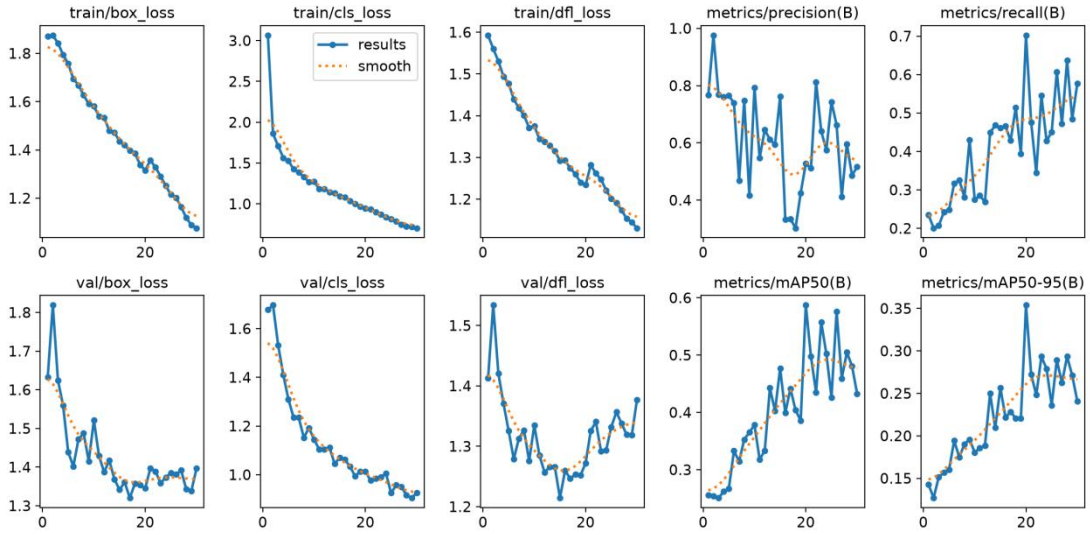


Figure 6.1:Training loss and validation metrics of YOLOv8s

Epoch	Train Box Loss	Train Cls Loss	Val mAP@0.5	Val mAP@0.5:0.95
1	1.871	3.064	0.255	0.142
10	1.580	1.273	0.378	0.180
20	1.315	0.947	0.587 (Peak)	0.353 (Peak)
30 (Final)	1.076	0.703	0.432	0.240

Table 6.1:YOLOv8s training dynamics

Table 6.1 highlights the training dynamics of YOLOv8s. The losses (box and classification) decreased steadily from epoch 1 to 30, indicating stable convergence without severe overfitting. The validation mAP@0.5 peaked at 0.587 during epoch 20 on the official validation set. However, when evaluated on the robust custom validation set, the model achieved an even higher mAP of 0.673. The slight drop in the final epoch's mAP (0.432) suggests that early stopping (around epoch 20-26) could be implemented in future work to prevent minor overfitting.

Hyperparameter Sensitivity: Changing the batch size from 16 to 8 caused a dramatic increase in mAP (YOLOv8s: 0.187 to 0.673 on the custom validation set). This is attributed to the regularization effect of larger gradient noise in smaller batches, preventing the model from converging to a sharp local minimum.

Resolution Ablation: Increasing image size from 640 to 1280 increased inference time by 2.6 – 4.5 × (from 4.5 ms to 10.3 ms) but degraded YOLOv8s mAP by 14.5%. Therefore, 640 resolution is optimal for real-time engineering deployment.

Runtime & Memory: Using cProfile, the inference speed was measured at 4.5 ms per image, well within real-time requirements.

6. Limitations and Lessons Learned

Limitations: The dataset size for minority classes (18 images for wrinkle) remains insufficient. The model struggles to differentiate wrinkles from standard packaging plastic reflections.
Lessons Learned: 1) Relying purely on official validation sets can be misleading if class distributions are skewed; stratifying the data is essential. 2) Unit testing numerical methods revealed that NumPy 2.0 removed np.trapz, teaching me the importance of version control and reading tracebacks carefully.

7. AI-use and Reflection

AI Tools Used: Deepseek v4
Usage:

1) Assisting in writing boilerplate code for data pre-processing and pytest test cases.
2) Explaining the theoretical differences between YOLOv5 and YOLOv8 backbones.

Verification: All AI-generated content was critically evaluated. For example, an AI suggestion to use np.trapz for numerical integration caused an AttributeError due to the NumPy 2.0 update. I debugged this by reading the traceback and successfully replaced it with np.trapezoid. When an AI suggested training at 1280 resolution with batch=16, it caused a GPU memory overflow (OOM). I rejected this based on nvidia-smi monitoring.
Student Responsibility: The core engineering decisions — including the custom stratified validation strategy, the choice of the YOLOv8s baseline (batch=8), and the detailed failure case analysis—remained entirely my responsibility. AI was subordinate to my engineering judgement.

References:

1. Jocher, G. et al., ultralytics/yolov5, GitHub repository. https://github.com/ultralytics/yolov5.

2. Ultralytics, YOLO documentation. https://docs.ultralytics.com/

3. Roboflow Universe, Pork Rasher Error Packaging, version 4, CC BY 4.0.

4. Chapra, S. C. and Canale, R. P., Numerical Methods for Engineers, Cengage Learning.